

## IT 360 – Final Project Report

### `bash_bulk_extractor.sh`

**Authors: Garrett Fraley & Marcellus Neal**

#### Introduction

Digital forensics investigations often begin with a triage phase, where the goal is to quickly identify high-value artifacts—such as credentials, email addresses, browser activity, and login history—without performing a full, time-consuming disk image analysis. Many professional tools (e.g., `bulk_extractor`) automate this process but can be complex to deploy or unavailable in constrained environments.

For this project, we developed `bash_bulk_extractor.sh`, a lightweight forensic triage script that mimics core bulk extraction capabilities using only standard Linux utilities. The script scans a specified target directory (such as a mounted drive or user home folder) for key forensic artifacts and consolidates the results into a structured output folder with integrity hashing. The design reflects core IT 360 principles: minimal impact, repeatability, transparency, and preservation of evidence integrity.

#### Purpose of the Tool

The goal of `bash_bulk_extractor.sh` is to provide a portable, easy-to-run triage tool capable of:

- Extracting email addresses and password-related strings
- Collecting login history and shell activity
- Carving URLs from browser history databases
- Generating summary reports and SHA-256 hashes for integrity verification

The tool is designed to run with root privileges to ensure full access to relevant artifacts while keeping the source data read-only. Although optimized for Ubuntu Linux, it relies only on standard commands such as `grep`, `last`, `lastb`, `strings`, `find`, and `sha256sum`, ensuring compatibility across many forensic environments.

#### Technical Design & Implementation

##### 1. Configuration & Pre-Flight Checks

At startup, the script performs the following:

- Accepts an optional `TARGET_DIR` argument, defaulting to `/home`
- Creates a timestamped `OUTPUT_DIR` in the working directory

- Initializes an extraction\_errors.log file
- Verifies the script is run as root
- Confirms that the target path exists

These checks ensure the tool can safely and reliably process the intended evidence source.

## 2. Module 1 – Bulk Extractor Mimic (Pattern Matching)

This module performs recursive pattern matching to approximate bulk extraction behavior:

- Email extraction using a regex pattern with `grep -r -a -b -o`
- Password keyword extraction using `grep` to identify credential-like entries

All errors encountered during recursion are stored in extraction\_errors.log.

## 3. Module 2 – User Activity (Logs & History)

This module gathers high-value user activity artifacts:

- Successful login history (wtmp) using `last -f`
- Failed login attempts (btmp) using `lastb -f`
- Aggregated bash shell history from all .bash\_history files located within the target directory

If target copies of wtmp and btmp are unavailable, the script falls back to system logs in /var/log.

## 4. Module 3 – Browser Artifacts (URL Carving)

The script searches for:

- Chrome/Chromium “History” files
- Firefox “places.sqlite” databases

For each matching file, the script:

- Logs processing details
- Runs strings to extract printable text
- Uses regex matching to carve URLs

Results are saved in carved\_urls.txt.

## 5. Module 4 – Integrity & Reporting

This module finalizes the collection:

- Produces a summary report (report\_summary.txt) containing timestamps, target path, and file listings
- Computes SHA-256 hashes of all output files and stores them in SHA256SUMS.txt
- Provides instructions for hash verification using sha256sum -c

### Testing & Validation

Testing was conducted in an Ubuntu Linux virtual machine. The script was executed with root privileges and tested with both default and custom target directories. Key observations:

- bulk\_emails.txt successfully captured email patterns from text and binary files
- bulk\_passwords.txt captured credential-like string patterns
- Login history artifacts were collected from either the target logs or fallback system logs
- Browser URL carving successfully extracted HTTP/HTTPS URLs from synthetic Firefox and Chrome data
- The SHA256SUMS.txt file allowed successful post-collection integrity verification

The script demonstrated stable behavior across multiple runs, recovering useful triage artifacts without interruption, even when some logs were missing.

### Analysis & Justification

bash\_bulk\_extractor.sh embodies several key digital forensics principles:

- Integrity – SHA-256 hashing prevents post-collection modification of evidence
- Repeatability – deterministic behavior and verbose logging make results replicable
- Minimal impact – read-only data collection avoids contaminating target artifacts
- Transparency – all actions and errors are recorded in log files

The tool prioritizes high-value triage artifacts that support early investigative leads. Its reliance on native Linux tools ensures auditability and forensic defensibility.

### Conclusion

bash\_bulk\_extractor.sh provides an effective, lightweight forensic triage solution suitable for academic and real-world investigative contexts. It automates the extraction of critical artifacts, structures output for review, and secures the results with integrity hashing.

The project strengthened our understanding of Linux internals, forensic triage workflows, and the role of automation in digital investigations. `bash_bulk_extractor.sh` serves as a solid foundation for future enhancements, such as browser database parsing, extended credential searching, or integration into a broader DFIR scripting toolkit.